KV-CACHE INFRASTRUCTURE OPTIMIZATION FOR LLM INFERENCE SYSTEMS

by

YUNXUAN ZHANG

Department of Computer Science and Engineering
The Hong Kong University of Science and Technology
Supervised by Prof. Kai Chen

October 2026, Hong Kong

TABLE OF CONTENTS

Title Page i

Table of Contents ii

Abstract iv

Chapter 1 Introduction 1

Chapter 2 Background and Problems 3

2.1 LLM Inference and the KV-Cache 3

2.1.1 Attention and Cached Keys and Values 3

2.1.2 Prefill, Decoding, and Reuse 4

2.2 Three Challenges in KV-Cache Management 5

2.2.1 Capacity Pressure 5

2.2.2 Access Bandwidth Bottleneck 6

2.2.3 State Preparation Overhead 7

2.3 A Framework for Classifying KV-Cache Infrastructure 8

2.3.1 Representation and Compression: What Is Retained? 8

2.3.2 Storage and Access: How Is It Stored and Retrieved? 8

2.3.3 Reuse and Execution: Who Uses the Cache? 8

Chapter 3 KV-Cache Infrastructure Techniques 10

3.1 Representation and Compression 10

3.1.1 Model Structural-Based Reduction 10

3.1.2 Selective Retention 13

3.1.3 Numerical Quantization 14

3.1.4 Comparison and Composability 15

3.2 Storage and Access 16

3.2.1 Allocation and Address Mapping 17

3.2.2 Storage Hierarchy and Data Movement 18

3.2.3 Comparison 20
3.3 Reuse and Execution 21
3.3.1 Reuse Scope and Matching 21
3.3.2 Cache Retention and Restoration across Requests 23
3.3.3 Cache-aware Execution Coordination 25
3.3.4 Comparison 26

Chapter 4 Discussion and Future Work 28
4.1 System-wide Trade-offs and Co-design 28
4.2 Ultra-Long Contexts 28
4.3 Long-lived Agentic Tasks 29

Chapter 5 Conclusion 31

References 32

KV-CACHE INFRASTRUCTURE OPTIMIZATION FOR LLM INFERENCE SYSTEMS

by

YUNXUAN ZHANG

Department of Computer Science and Engineering

The Hong Kong University of Science and Technology

ABSTRACT

Large language models (LLMs) use a key-value cache (KV-cache) to avoid recomputing historical attention states during inference. Longer contexts and higher request concurrency make cache management a major constraint on efficiency. This survey reviews KV-cache infrastructure, covering the mechanisms that represent, store, access, and reuse cached keys and values. We organize the survey into three categories: representation and compression, storage and access, and reuse and execution management. We examine how these techniques address capacity pressure, access bandwidth bottleneck, and the overhead of constructing or restoring cached state. The review compares architectural changes, compression, memory management, offloading, and reuse in terms of their benefits, costs, and applicability. We emphasize that optimizations can shift bottlenecks and introduce processing, communication, or quality trade-offs, motivating coordinated design across model and system components. Finally, we discuss future requirements for ultra-long contexts and long-lived agentic tasks, including efficient history access and state preservation across tool interactions.

CHAPTER 1

INTRODUCTION

Large language models (LLMs) have made significant advances and support various applications such as conversational assistants, document analysis, and code generation. LLMs generate text autoregressively, using information from preceding text (the prompt and previously generated tokens) to compute the next output token. Modern LLM inference systems commonly use a *key-value cache* (KV-cache) to accelerate token generation. At each generation step, the model reuses the cached keys and values of preceding tokens stored in memory for attention computation, avoiding the computational cost of recomputing these states [11].

Storing, accessing, and managing the KV-cache are key concerns in LLM inference systems. First, the KV-cache consumes substantial memory [11, 18]. Limited GPU memory severely restricts the number of requests that can run concurrently and can leave computational resources underutilized. This pressure is further intensified by the demand for longer context windows—reaching one million tokens in some models—to support complex tasks [32]. For a fixed model and cache format, KV-cache size grows linearly with sequence length, making its storage increasingly demanding. Its large size also makes repeated cache access during token generation memory-intensive [18]. Finally, requests in applications such as multi-turn conversations, question answering over articles, and collaboration among agents can share substantial portions of their context. Locating reusable KV-cache entries, scheduling requests to use them, and sharing them across requests remain major challenges for modern LLM inference systems [39, 23].

In this survey, we use *KV-cache infrastructure* to denote the methods and system components that represent, store, access, and manage cached keys and values during LLM inference, the process of running a trained model to produce outputs. This scope includes techniques that reduce the amount of cached data, mechanisms that organize and move it across memory and storage devices, and policies that determine when to retain, reuse, or

discard it. It also includes decisions about where requests run when those decisions depend on the availability of cached data. Together, these functions connect the cache used by the model with the resources needed to support it.

We organize the literature around three questions:

- What information should the cache retain and in what form;
- How should that information be stored and accessed;
- When and by which requests should it be reused?

These questions cover changes to the model's cache requirements as well as cache management within and across servers. For each group of techniques, we review representative designs, compare their benefits and costs, and discuss the conditions under which they are useful.

The remainder of this survey is organized as follows. Chapter 2 explains how the KV-cache works and introduces the management challenges and classification framework. Chapter 3 reviews existing techniques and systems within this framework. Chapter 4 discusses open research challenges, and Chapter 5 concludes the survey.

CHAPTER 2

BACKGROUND AND PROBLEMS

This chapter introduces how the KV-cache supports LLM inference, explains the main challenges in managing it, and establishes the framework used to review existing techniques. We focus on autoregressive Transformer models, which generate an output sequence one token at a time.

2.1 LLM Inference and the KV-Cache

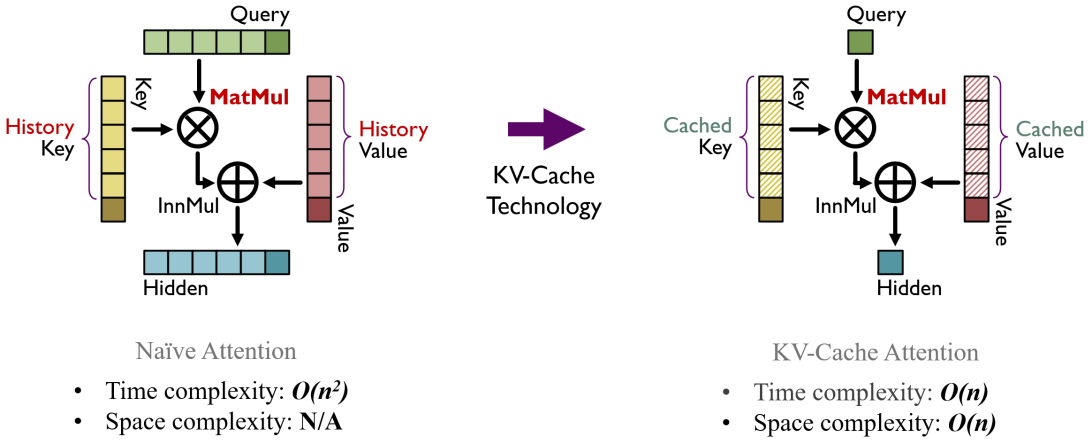


Figure 2.1. KV-cache technology trades quadratic computing time complexity by linear HBM space complexity.

2.1.1 Attention and Cached Keys and Values

A Transformer processes tokens through a sequence of layers. Within each layer, the attention operation combines information from different token positions. It uses three vectors for each token: a *query*, which determines what information to retrieve; a *key*, which is compared with the query to determine relevance; and a *value*, which carries the information to be combined. These vectors are computed from the token's representation at that layer using learned transformations [29].

For one attention head, let q_t, k_t, and v_t denote these vectors at position t. An attention head is one of several parallel attention operations within a layer. With causal attention, a position can use only its own and preceding positions. Its attention output is

$$o_t = \mathrm{softmax}\left(\frac{q_t K_{1:t}^{T}}{\sqrt{d}}\right) V_{1:t}, \tag{2.1}$$

where $K_{1:t}$ and $V_{1:t}$ collect the keys and values through position t, and d is the dimension of a key vector. The softmax operation converts the query–key scores into weights for combining the values.

Appending a token does not change the earlier positions' keys and values under causal attention. The system can therefore retain them at every layer, compute only the new token's keys and values, and append these to the cache. Historical queries do not need to be retained for subsequent attention steps. The KV-cache avoids the recomputation of historical Key and Value entries [11].

2.1.2 Prefill, Decoding, and Reuse

Inference usually consists of two phases: prefill and decoding. During *prefill*, the model processes the prompt, constructs its KV-cache, and produces the first output token. The prompt tokens are already known, so their computations can be performed in parallel within each layer. During *decoding*, the model processes each newly generated token to produce the next one, reading and extending the cache at every step. Successive decoding steps depend on earlier outputs and therefore proceed in an auto-regressive pattern [40].

Although decoding proceeds sequentially within each request, independent requests can be processed together in a *batch*. For example, one decoding iteration can process one new token from each of several active sequences in parallel, improving GPU utilization. These sequences share the model weights, but each requires its own attention history. Their KV-caches therefore coexist in memory and may have different lengths, depending on the requests' prompt lengths and generation progress [11].

The cache may remain useful after a request finishes. A later conversation turn can reuse the history already processed, and different requests may share an initial sequence

of tokens, called a *prefix*. Retaining the cache for that prefix can avoid redundant computation [39]. Exact reuse requires compatible model parameters, token prefixes, positions, and attention settings. Identical text appearing after different preceding text does not generally have identical keys and values, because its representations depend on that preceding text. Thus, overlapping documents do not automatically imply that all corresponding cache entries can be shared unchanged.

2.2 Three Challenges in KV-Cache Management

2.2.1 Capacity Pressure

The cache occupies memory throughout generation and grows as more tokens are processed. Assuming that all layers have the same number of key-value heads and that keys and values have equal dimensions and precision. The total cache size is

$$M_{KV} = 2LH_{KV}ds\sum_{i=1}^{B} T_i, \tag{2.2}$$

where L is the number of layers, H_{KV} the number of key-value heads per layer, d the dimension of each key or value vector, s the bytes per element, B the number of cached sequences, and T_i the cached length of sequence i. The factor of two accounts for both keys and values. This expression assumes separate storage for each sequence and excludes allocation overhead and metadata.

For example, Llama 3.1 8B has 32 layers, 8 key-value heads per layer, and a head dimension of 128 [19]. With the KV-cache stored in 16-bit floating-point format (two bytes per element), Equation 2.2 gives 4 GiB for one sequence with 32,768 cached tokens. A batch of 16 such sequences requires 64 GiB of KV-cache, before accounting for model weights and temporary buffers. This example shows how context length and concurrency jointly increase memory demand. The number of key-value heads also matters: grouped-query attention allows multiple query heads to share a key-value head, reducing the cache size [1].

Limited GPU memory can restrict the number of requests processed together and prevent the system from fully using its computational resources. Unused space within al-

locations and duplicate copies of shared prefixes further reduce effective capacity [11]. Moving cached data to CPU memory or storage devices can expand capacity, but requires bringing that data back or accessing it through a slower connection when needed.

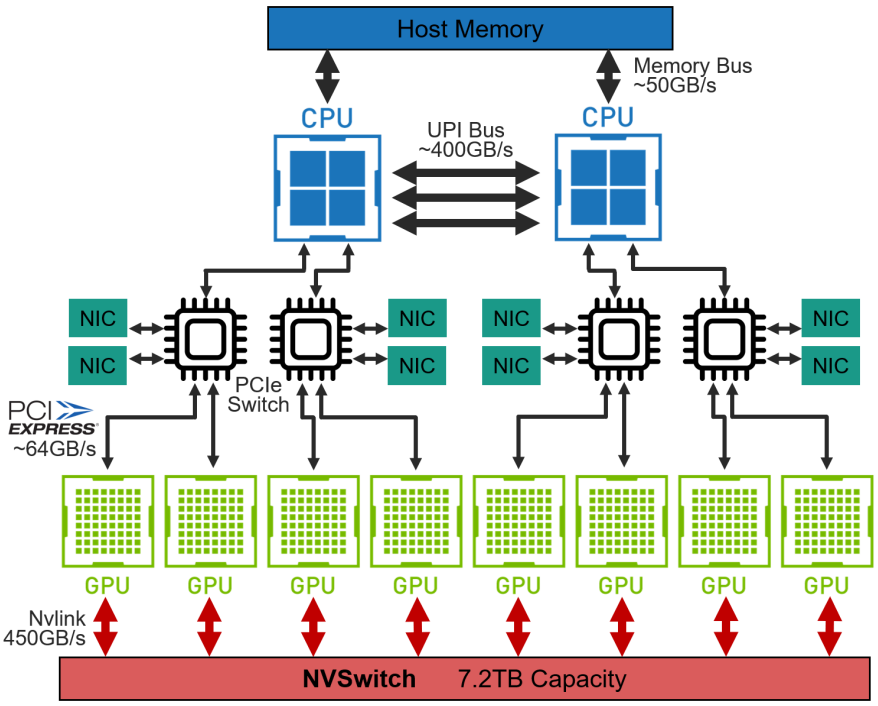


Figure 2.2. The memory hierarchy and interconnect fabric topology of a typical GPU server. Multiple GPUs is interconnected by high-speed scale-up fabric. Host memory is connected to GPU by slower PCIe fabric. GPU can also access remote memories via NICs.

2.2.2 Access Bandwidth Bottleneck

Fitting the cache in memory does not ensure fast attention. With dense attention, each decoding step uses the keys and values of all preceding positions. As the sequence grows, the amount of historical data accessed per step also grows. GPU computation can consequently be limited by the rate at which memory delivers this data, rather than by arithmetic throughput [18].

Capacity and bandwidth are related but distinct. A storage arrangement may accommodate more cached tokens without reducing the bytes read for each generated token. Conversely, an attention implementation may reduce data transfers without changing the stored cache size. The importance of cache traffic depends on context length, the number

of requests processed together, attention architecture, and hardware; model-weight traffic and other computation also contribute to decoding time.

When cached data resides outside GPU memory, access additionally depends on the connections between devices or servers. The relevant question is therefore both how much data an execution needs and how quickly the storage system can deliver it. Techniques that reduce cache size can help both capacity and bandwidth, but their conversion or reconstruction costs must also be considered.

2.2.3 State Preparation Overhead

Before an execution can use a cache, the required keys and values must be available in a usable form. We call the time spent constructing, loading, or restoring them *state preparation overhead*. A new prompt requires cache construction through prefill. A repeated prompt may instead use retained entries, while a paused request may need entries reloaded from another device. If prefill and decoding run on different servers, the cache must also be transferred between them [40, 23].

Reuse saves computation only when finding and retrieving the retained data is worthwhile. Its benefit depends on how much computation is avoided, where the data resides, and the time required to transfer and prepare it. Retaining more entries also consumes capacity that could serve active requests. Systems must therefore decide which entries to keep and whether to retrieve or recompute them when needed [23].

These three challenges are analytical perspectives rather than independent resource categories. State preparation can itself be limited by computation or transfer bandwidth. We distinguish it from access bandwidth pressure to separate the cost of making a cache ready from the recurring cost of using it during generation. An optimization may relieve one challenge while worsening another, such as expanding capacity through remote storage while increasing preparation time.

2.3 A Framework for Classifying KV-Cache Infrastructure

We classify techniques by the decision they primarily change. The following three categories apply to both single-server and distributed systems. The challenges in Section 2.2 provide a common basis for analyzing methods within every category.

2.3.1 Representation and Compression: What Is Retained?

This category concerns the information represented by the cache and the space required to encode it. It includes attention architectures that change which keys and values are produced, methods that retain only selected historical information, and quantization, which stores numerical values using fewer bits. These methods can reduce both storage and data access requirements. Their costs may include model changes, additional processing, or reduced output quality [1, 18].

2.3.2 Storage and Access: How Is It Stored and Retrieved?

This category concerns how a chosen cache representation is mapped to memory and storage devices and delivered to computation. It includes allocation, data layout, address mapping, placement across devices, and transfer scheduling. For example, dividing the cache into separately allocated blocks can reduce wasted memory [11]. Moving entries to a larger storage tier can expand available capacity, while arranging transfers in advance can reduce waiting. The defining issue is the physical storage and access path, regardless of whether that path crosses a server boundary.

2.3.3 Reuse and Execution: Who Uses the Cache?

This category concerns how cached data is associated with requests and retained over time. It includes identifying reusable prefixes, deciding which completed requests' entries to keep, sharing entries across requests, and assigning requests to servers that hold useful cached data [39, 23]. It also covers coordination between execution stages when cache availability affects where or when those stages run. Its primary concern is selecting useful cached work and coordinating its consumers.

The categories distinguish responsibilities within a system, so a paper may contribute to more than one. For example, a mechanism for transferring a cache belongs to physical storage and access, whereas a policy that sends a request to a server holding its prefix belongs to reuse and execution management. Likewise, removing tokens from an active sequence to approximate attention changes its representation; discarding an inactive request's cache because it is unlikely to be reused is a retention decision. We use these distinctions to organize the review while discussing interactions between techniques explicitly.

Within this framework, we examine each technique in terms of the bottleneck it addresses, the mechanism it uses, the costs it introduces, and the conditions under which it is useful. This analysis connects design choices to their benefits and trade-offs, including cases where relieving one bottleneck increases pressure on another.

CHAPTER 3

KV-CACHE INFRASTRUCTURE TECHNIQUES

This chapter reviews techniques for representing and compressing the KV-cache, organizing its storage and access, and managing its reuse across executions. We use the three challenges introduced in Chapter 2 to explain the benefits and costs of these designs.

3.1 Representation and Compression

Representation and compression techniques reduce the data volume of KV-cache that must be retained for attention. Equation 2.2 suggests three broad approaches: reduce the number or dimensions of independent key-value representations, retain fewer historical entries, or use fewer bits for each value. We organize this section accordingly into structural compression, selective retention, and numerical quantization. The distinction concerns what is compressed; a system may combine several approaches.

3.1.1 Model Structural-Based Reduction

The model structure changes how keys and values are represented across heads, layers, or vector dimensions. Some methods introduce an attention architecture that produces a smaller cache from the outset. Others transform an existing model to use a more compact representation. This distinction matters because an architectural change may require training, whereas a transformation of an existing model must control the error introduced by compression.

Sharing across heads and layers. In conventional multi-head attention (MHA), each head uses separate projections to generate queries, keys, and values. Each head therefore requires its own cached keys and values for all retained token positions. The cache size grows with the number of heads as well as the sequence length.

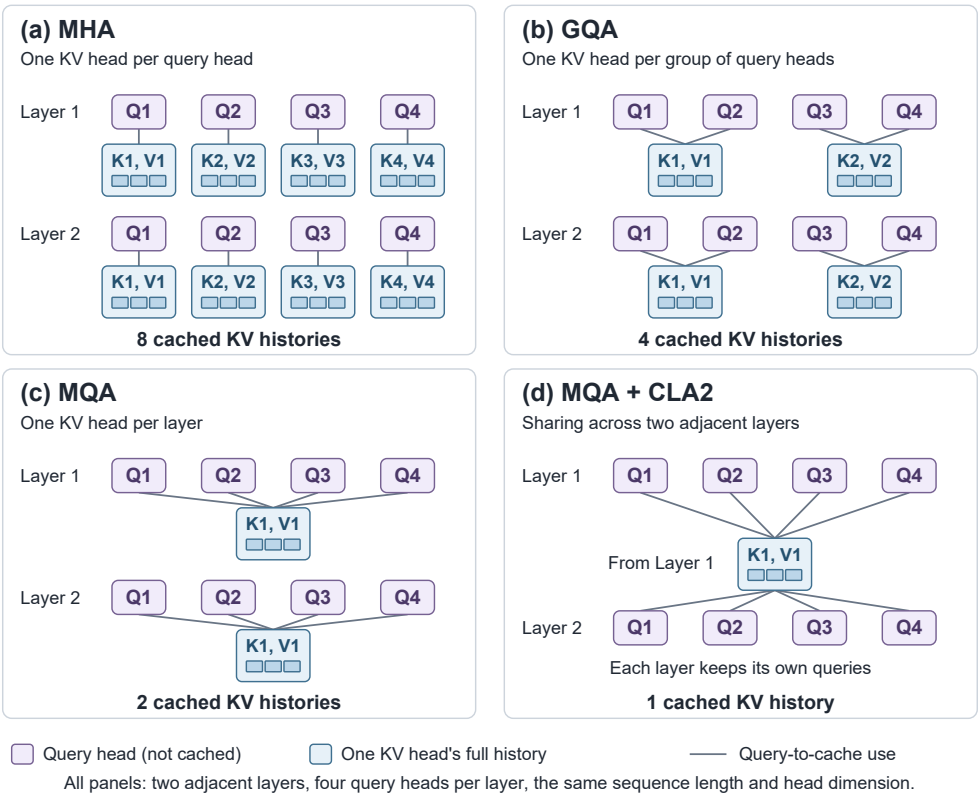


Figure 3.1. KV-cache sharing across heads and layers.

Grouped-query attention (GQA) divides query heads into groups, with each group sharing one key-value head [1]. Multi-query attention (MQA), introduced to reduce decoding bandwidth demand, is the special case with only one group [24]. GQA reduces cached vectors without reducing the number of query heads. This distinction helps explain why sharing can preserve model quality: distinct queries can assign different attention weights to the same keys and extract different weighted combinations of the shared values. Keeping several KV groups also preserves more representational diversity than MQA. To adapt an existing MHA checkpoint, the authors average the key and value projection weights within each group and continue training. This allows the model to adjust to sharing, achieving quality close to MHA in their experiments. Quality preservation is therefore an empirical outcome of the sharing configuration and adaptation, rather than a guarantee of lossless compression.

Cross-Layer Attention (CLA) extends sharing to adjacent layers [3]. In its pairwise configuration, the first layer generates keys and values that both layers use, halving the

number of separately cached layers. The second layer still generates its own queries from its updated hidden states and retains its own output projection and feed-forward network. Mechanistically, this allows successive layers to extract different information from a common cache and continue transforming it; sharing KV does not require identical attention outputs or merging whole layers. Training with this architecture lets the shared representation serve both layers. The paper's experiments show that pairwise sharing can retain nearly the same accuracy as MQA while reducing its cache size. This is evidence that separately generated KV representations are not essential at every layer in the tested models, rather than proof that neighboring layers' KV tensors in an arbitrary pretrained model are already interchangeable.

MiniCache uses cross-layer similarity without retraining the model [14]. It merges similar KV vectors from adjacent middle-to-deep layers by interpolating their directions while retaining their magnitudes, and keeps sufficiently different entries separate. Whereas CLA learns a shared representation through its architecture, MiniCache approximates representations already produced by a model. Its capacity benefit therefore depends on the observed similarity and the fraction of entries that remain unmerged.

Using fewer dimensions. Multi-head Latent Attention (MLA), introduced in DeepSeek-V2, instead builds a compact representation into the model architecture [5]. It generates a shared latent vector—a smaller learned representation—from which multiple key and value heads are defined. During decoding, transformations can be combined with the query and output projections to avoid explicitly reconstructing all historical keys and values. MLA also separates the positional component of attention and retains an additional positional key. Its cache therefore contains both the latent vectors and this positional information. MLA is learned as part of the model architecture.

These methods primarily address capacity pressure. They can also reduce access bandwidth pressure when the attention implementation consumes shared or reduced representations efficiently. However, a smaller stored cache does not imply a proportional reduction in memory traffic: shared data may still be read by multiple operations. Architectural adaptation, approximation error, and support in the attention implementation determine whether a structural reduction becomes a practical serving benefit.

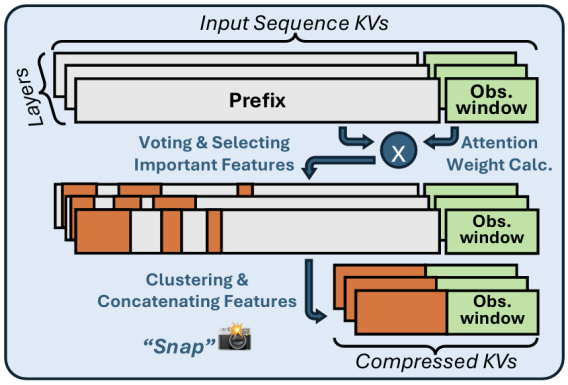


Figure 3.2. KV-cache selection.

3.1.2 Selective Retention

Selective retention reduces the historical information kept for an active sequence. A cache budget specifies how many entries may remain, and a selection policy determines which entries to retain as the sequence grows. The main design choices are how importance is estimated, how the budget is distributed across heads and layers, and when the selection is updated.

Retention by observed importance. H_2O uses observed attention to select useful historical entries [38]. It combines recent tokens with *heavy hitters*, tokens that have accumulated large attention scores. As generation proceeds, the system updates these statistics and removes lower-priority entries to respect the cache budget. Compared with a fixed positional rule, this can preserve older information that continues to receive attention. The cost is maintaining importance scores and performing selection during inference. Moreover, accumulated attention reflects past queries, so it cannot guarantee that an evicted token will remain irrelevant to future queries.

SnapKV moves the selection of historical prompt entries to the end of prefill [13]. It observes attention from a short window at the end of the prompt to estimate which earlier positions each head will need during generation. It pools importance scores over neighboring positions, selects entries for each head, and retains the observation window. The resulting prompt cache is smaller before decoding begins. This reduces the need to repeatedly select among all prompt entries, but relies on the prompt's final attention pat-

tern being informative about subsequent generation. Selecting the prompt cache also does not, by itself, impose a fixed bound on all entries added during a long output.

Distributing the retention budget. Choosing which entries to retain is separate from deciding how much capacity each layer receives. PyramidKV observes that attention in its studied models is more broadly distributed in lower layers and more concentrated in higher layers [4]. It assigns larger budgets to lower layers and smaller budgets to higher layers, while selecting important positions within the heads. The allocation reflects an observed pattern rather than a universal property of attention; its suitability depends on the model and workload.

Selective retention can relieve both capacity pressure and access bandwidth pressure because fewer entries remain for subsequent attention. Its principal trade-off is loss of historical information, together with the cost of computing and maintaining selection decisions. These costs occur at different times: prompt selection adds work during state preparation, while an online eviction policy adds work during generation. If discarded information is needed again, restoring it may require recomputation from the original context. The useful comparison is therefore between the retained information, the timing of selection, and the demands of the target task, rather than cache size alone.

3.1.3 Numerical Quantization

Quantization stores numerical values using a smaller set of representable values, reducing the bits needed per cache element. Unlike token eviction, it can retain entries for every historical position, but approximates their contents. Its effectiveness depends on which values share quantization parameters, how unusual values are handled, and how the compressed representation participates in attention.

KIVI uses different quantization rules for keys and values [18]. A *channel* is one dimension of a key or value vector; a quantization group contains numbers that share quantization parameters. For keys, KIVI groups numbers from the same channel across tokens. Large key magnitudes often concentrate in particular channels, so this prevents them from reducing precision in other channels. For values, it groups numbers within each token's

vector, limiting the influence of one token's extreme values on other tokens. KIVI keeps recent entries at higher precision and quantizes older entries in fixed-size groups, since key quantization requires accumulating multiple tokens. The cache therefore contains both low-precision groups and a higher-precision recent portion, rather than storing every entry in two bits.

Coupled Quantization (CQ) divides each key vector and each value vector separately into small, non-overlapping groups of consecutive components [37]. Across tokens, components within a group exhibit statistical dependencies: some combinations of values occur more often than others. CQ learns a collection of representative vectors, called a *codebook*, for each group and stores only the index of the representative that approximates its components together. Keys and values are not combined into one group. One index thus encodes several components. Codebooks are learned from sample inputs, accounting for how quantization errors affect model outputs. This approach requires codebook storage, work to select representative vectors, and specialized attention computation.

Quantization directly reduces capacity demand and can reduce the bytes transferred during cache access. Whether it improves generation speed depends on how KV-cache are decoded and consumed. *Dequantization* converts a compressed representation back to forms usable by the computation. The extra writes, reads, and temporary storage during dequantization can offset compression benefits. Numerical error, metadata, higher-precision exceptions, and conversion work must therefore be considered alongside nominal bit width.

3.1.4 Comparison and Composability

Table 3.1 summarizes the main distinctions among the reviewed methods. Architectural methods change which representations a model produces; selective retention changes which historical entries survive; quantization changes how retained values are encoded.

The approaches can be complementary: a model with shared KV heads may retain selected token positions and quantize the remaining values. Compatibility is nevertheless a design question. A selection policy must respect any sharing of the underlying entries; a quantizer calibrated for full-dimensional keys may need to be adapted to latent vectors.

Table 3.1. Representative compression methods and their design trade-offs. Calibration configures a method using sample inputs; it need not involve updating the model through training.

Method	Main change	Preparation or adaptation	Main cost or limitation
GQA [1]	Share KV heads	Training or checkpoint adaptation	Fewer independent KV heads
CLA [3]	Share KV across layers	Architectural training	Dependencies between layers
MiniCache [14]	Merge similar adjacent-layer KV	No retraining	Approximation; unmerged exceptions
MLA [5]	Cache a latent representation	Architectural training	Specialized attention computation
H_2O [38]	Keep recent and high-score tokens	Online score updates	Selection cost; future relevance uncertain
SnapKV [13]	Select prompt entries per head	Attention observation at prefill	Depends on observed prompt attention
PyramidKV [4]	Vary retention budgets by layer	Layer budget configuration	Assumes suitable attention patterns
KIVI [18]	Quantize keys and values differently	Online grouped quantization	Numerical error; residual cache
CQ [37]	Encode channels jointly	Codebook calibration	Codebook storage and lookup

Errors from selection, dimensional reduction, and quantization can also interact.

Across the three bottlenecks, compression most directly targets capacity and access bandwidth. A smaller representation may also reduce transfer or loading time when cached data is reused, while encoding, selection, or restoration may add state preparation work. This motivates treating representation as one part of the infrastructure: the storage system must support the chosen format, and reuse policies must account for what information the compressed cache still contains.

3.2 Storage and Access

Storage and access techniques determine how KV-cache entries occupy physical memory and reach attention computation. We review allocation and address mapping together with their support for attention, followed by storage across devices. These mechanisms support a chosen cache representation; identifying reusable contexts and coordinating their consumers are separate responsibilities.

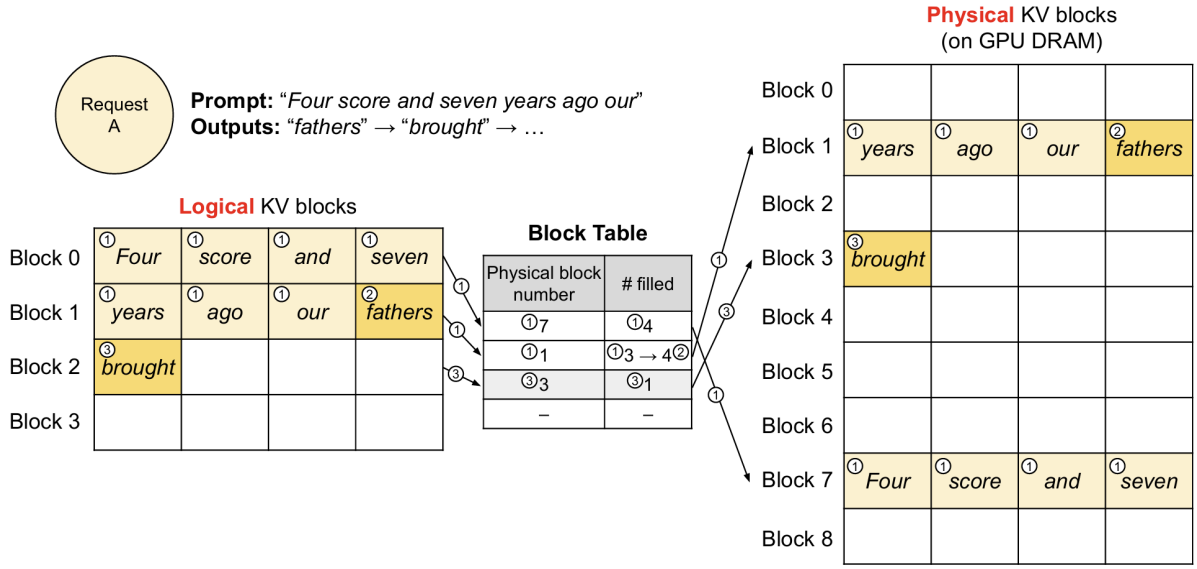


Figure 3.3. PagedAttention and vAttention divide the KV-cache of a sequence into fixed–sized blocks (pages) and manage them in discrete physical memory slots.

3.2.1 Allocation and Address Mapping

Sequences grow during decoding and have different lengths. Reserving a large contiguous region for each sequence wastes unused space, while repeatedly enlarging an allocation may require copying existing entries. Dynamic allocation must therefore accommodate growth without excessive reservation or movement.

PagedAttention [11] allocates fixed-size cache blocks as a sequence grows. A block table maps positions in the sequence to blocks in GPU memory, allowing attention to read physically scattered data. This reduces wasted capacity, although the final block may remain partially unused. The cost is maintaining mappings and adapting attention implementations to follow them. Multiple sequences can also reference the same physical blocks; deciding which contexts qualify for sharing is a separate reuse problem.

vAttention instead preserves contiguous *virtual addresses*, the addresses used by the program, while allocating the underlying physical memory on demand [22]. It uses GPU virtual-memory support to separate these two views, allowing existing attention implementations to retain contiguous access. Both approaches relieve capacity pressure; their main distinction is whether address mapping is handled explicitly by the attention implementation or through the virtual-memory system. The latter still requires managing

allocation and mapping costs.

Supporting attention computation. Attention routines must support the cache layout. For example, an implementation requiring contiguous data would need scattered cache blocks copied into a temporary buffer, adding memory traffic and space. PagedAttention avoids this copy by reading through block mappings. FlashInfer supports different organizations, including paged and shared-prefix caches, through a common block-based representation and corresponding GPU attention routines [34]. These routines access the stored blocks directly, but must be compatible with the chosen layout. Efficient allocation therefore needs matching attention support to avoid additional access costs.

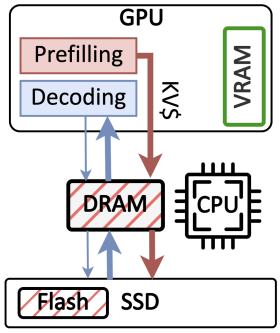


Figure 3.4. Offloading KV-cache to downstream memory tiers can improve the KV-cache capacity, but may introduce KV-cache transfer bottleneck over slow interconnect fabrics.

3.2.2 Storage Hierarchy and Data Movement

Expanding capacity. A storage hierarchy combines devices with different capacities and access costs. *Offloading* places data outside GPU memory to free space for active computation. FlexGen coordinates tensor placement and access across GPU memory, CPU memory, and disk [25]. It also supports delegating attention computation to the CPU when avoiding cache transfers outweighs slower execution. These choices support larger batches under limited GPU capacity, but its throughput-oriented design can increase response time.

Selecting accesses without deleting history. Reducing the entries read at each step is distinct from permanently evicting them. QUEST uses the current query and per-channel

minimum and maximum keys in each cache page to estimate page importance, then reads selected pages for attention [28]. Other pages remain available for later queries. This primarily relieves access bandwidth pressure rather than total storage demand, and introduces selection work and attention approximation. ShadowKV combines selective access with capacity reduction: it retains a low-rank key representation, offloads values, and reconstructs selected keys while fetching the corresponding values [27]. Its benefit depends on balancing reduced transfers against selection, reconstruction, and approximation costs.

Co-design with sparse attention. HiSparse [30] connects sparse attention with hierarchical storage. Sparse attention selects only a subset of historical positions at each step, but future selections may still require the full history. HiSparse keeps this history in CPU memory and maps selected positions to a bounded set of GPU cache slots, fetching only missing entries. Compact state used to select positions remains on the GPU. For models that share selections across layers, later layers' accesses are known early, enabling prefetching during earlier computation. This is a co-design opportunity: model-level sharing exposes future accesses to the storage system. HiSparse changes placement without adding approximation to the underlying sparse attention; its costs are cache management and transfers on misses.

Reducing transfer overhead. InfiniGen predicts important KV entries for a subsequent layer and fetches them from CPU memory ahead of use, a technique called *prefetching* [12]. Selective fetching reduces traffic and can overlap transfers with computation. It adds prediction work and may introduce attention error: entries omitted from a computation do not contribute to its output, even if they remain stored elsewhere. This connects access optimization with the selective methods in Section 3.1.2.

Moving data or moving computation. NEO offloads part of both the cache and attention computation to the CPU [10]. Processing entries where they reside avoids repeatedly transferring the full offloaded history. Its pipeline overlaps CPU and GPU work, while load-aware scheduling balances their responsibilities.

The central trade-off is between transferring KV data for GPU execution and comput-

ing near CPU-resident data. The former can be limited by transfer bandwidth; the latter reduces traffic but may bottleneck on CPU computation or memory bandwidth. Systems must balance these costs according to the volume of accessed data, hardware capabilities, and current load.

DirectKV uses fast links between CPUs and GPUs to change this access path [20]. On platforms such as GH200, GPU attention routines directly read CPU-resident KV instead of first copying it into a GPU staging buffer. This saves temporary GPU capacity and avoids redundant movement, but still consumes interconnect bandwidth. It requires hardware and attention routines suited to direct CPU-memory access; zero-copy does not imply zero communication.

Encoding for network transfer. CacheGen reduces context-loading delay by encoding KV tensors into compact bitstreams and adapting compression levels to available network bandwidth [17]. Its objective differs from minimizing resident GPU cache size: encoding and decoding must be inexpensive enough that reduced transmission time produces a net benefit. Stronger compression can reduce preparation delay on a slow link while increasing approximation error. This connects representation choices to the conditions under which cached state is delivered.

3.2.3 Comparison

Table 3.2 summarizes the main benefits and costs. Allocation reduces wasted capacity; offloading expands available capacity; access optimization reduces traffic or waiting. These benefits are distinct: paging does not by itself reduce the historical entries required by dense attention, and a larger storage tier may increase state preparation time.

These mechanisms complement compression when storage and attention support the chosen format. Reuse management determines which cached entries to retrieve or recompute.

Table 3.2. Representative storage and access mechanisms and their principal trade-offs. Benefits depend on the workload and hardware.

Method	Main benefit	Main cost or limitation
PagedAttention [11]	Less allocation waste	Block mappings; adapted attention kernels
vAttention [22]	Dynamic physical allocation	Virtual-memory mapping and allocation costs
FlashInfer [34]	Efficient access to varied layouts	Layout-specific attention support
FlexGen [25]	Larger effective capacity	Transfers; latency-insensitive workload focus
QUEST [28]	Fewer pages read per query	Selection; attention approximation
HiSparse [30]	Bounded GPU KV residency	Sparse access required; miss transfers
InfiniGen [12]	Less transfer traffic and waiting	Prediction overhead; attention approximation
NEO [10]	More capacity; less cache movement	CPU resource demand; pipeline balancing
DirectKV [20]	No GPU staging copy	Fast interconnect; adapted attention routines
CacheGen [17]	Less network traffic	Encoding cost; compression error

3.3 Reuse and Execution

Reuse and execution management determines which requests can use existing KV-cache and how to coordinate their execution with cache availability. We organize these decisions around identifying reusable state, retaining it between requests, and assigning work to the resources that can use it. The main opportunity is to reduce state preparation overhead, while accounting for the capacity and bandwidth consumed by reuse.

3.3.1 Reuse Scope and Matching

Matching common prefixes. Under causal attention, a prefix's KV-cache does not depend on tokens that follow it. Requests with the same token prefix can therefore reuse its cache, provided the model, token positions, and attention configuration are compatible. SGLang implements this reuse through RadixAttention [39]. It indexes token sequences in a *radix tree*, a tree whose edges represent token segments and whose shared paths represent common prefixes. A new request follows these paths to find the longest cached prefix; only the remaining suffix requires prefill. This supports reuse across both concurrent requests and successive calls. Its scope is constrained by prefix identity: matching a passage that appears after a different preceding context is insufficient.

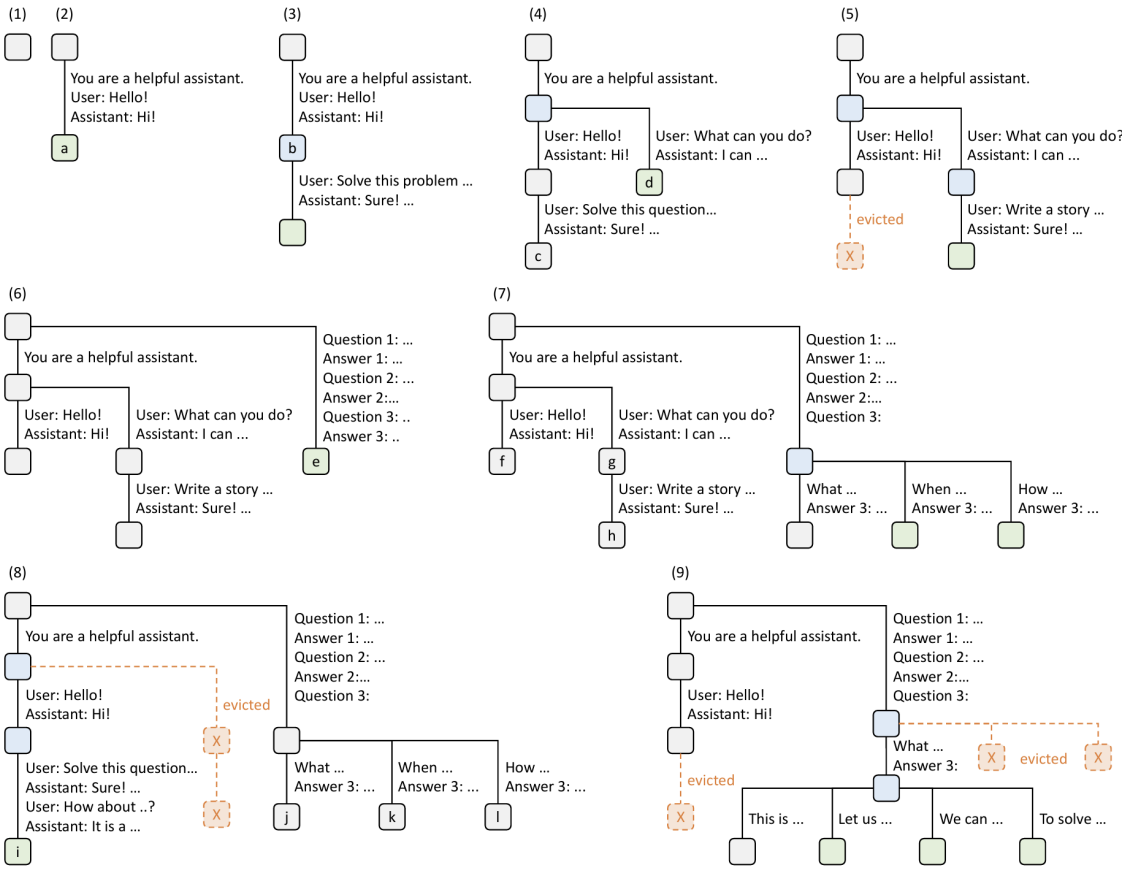


Figure 3.5. Prefix sharing through Radix Tree. Sharing KV-cache of the same segment of sequence can avoid re-prefill and save the memory space required for storing KV-cache.

Reusing non-prefix passages. Document question answering often combines retrieved passages in different orders. Concatenating their independently computed KV-cache introduces two problems. First, *positional encoding* must match the new input: cached keys may encode a passage's original token positions rather than its positions after concatenation, producing incorrect attention scores unless corrected. Second, *causal dependencies in the attention forward pass* are missing. For an input consisting of passage A followed by passage B, tokens in B can attend to preceding tokens in A. Computing B independently omits these interactions. Its attention outputs, and consequently the KV generated in subsequent layers, can therefore differ from full prefill of the combined input. Correcting positional encoding alone cannot reconstruct these missing computations.

CacheBlend observes that these dependencies affect tokens unevenly [33]. It measures *KV deviation*, the difference between cached and recomputed KV. Updating a small frac-

tion of high-deviation tokens can preserve generation quality in its experiments. This supports approximate reuse: skipped entries may still contain errors, but their effects can be small enough to tolerate.

The design first corrects cached keys for all shifted tokens using rotary positional encoding (RoPE), which encodes position through vector rotations. It then progressively filters tokens for recomputation across layers. Starting with all tokens, it uses differences between newly computed and cached KV to select candidates for subsequent layers, exploiting the observed correlation of deviations across neighboring layers. Later filtering considers only the remaining candidates; it does not first obtain full-prefill KV for every layer. Selected tokens recompute their queries, keys, values, and layer outputs, while attending to a mixture of updated and cached KV at all causally visible positions. Savings come from skipping other tokens' computation in later layers. This trades state preparation work against selection overhead and approximation error, without guaranteeing equivalence to full prefill.

Prompt structure offers another way to organize reuse. Prompt Cache lets applications declare reusable modules and their positional arrangements through a schema [8]. This makes reuse explicit, but positional consistency alone does not reconstruct attention dependencies between independently prepared modules. CacheSlide targets agent prompts whose reusable segments preserve their relative order while shifting in absolute position [15]. It combines a position-encoding scheme for these segments with partial attention computation and learned weights for mixing updated and cached KV. These methods exploit structured workloads to broaden reuse, with costs in application constraints, correction work, or learned configuration.

3.3.2 Cache Retention and Restoration across Requests

Completing a request does not necessarily end the usefulness of its cache. In a multi-turn conversation, the next request typically includes the previous interaction as a prefix. Keeping that state avoids rebuilding it, but competes with other cached contexts and active requests for capacity. Unlike removing tokens from an active sequence in Section 3.1.2, evicting an inactive cache sacrifices a future reuse opportunity rather than directly approximating the current attention computation.

SGLang retains prefixes after requests finish [39]. To reclaim space, it removes the least recently used eligible leaves of its radix tree. Shared ancestors remain reusable, while references from running requests protect entries still in use. Recency provides an inexpensive retention rule, although it cannot predict all future accesses.

Pensieve and CachedAttention retain conversation state across turns rather than discarding it after each response [35, 6]. Pensieve manages history across GPU and CPU memory and supports attention over non-contiguous cached data. CachedAttention coordinates fetching and eviction with hints from the inference scheduler, prioritizing state required by upcoming requests. Layer-wise pre-loading and asynchronous saving overlap access with computation. Both reduce repeated prefill by extending cache lifetime, while retaining history consumes storage and makes restoration part of request latency.

HCache introduces an intermediate option between loading complete KV and recomputing from tokens [7]. It saves the hidden states entering model layers and restores KV through their key and value projections, avoiding repeated attention and feed-forward computation. Its scheduler combines restoration methods to balance computation and I/O, while its storage organization reconciles incremental saving with bulk restoration. The capacity advantage depends on the model: hidden states need not be smaller than KV when few KV heads or latent representations are used. The broader principle is to choose both what state to retain and how to recover executable KV from it.

Retention should therefore consider both the likelihood of reuse and its cost. A large cache may save substantial prefill work but occupy space that could retain several other useful contexts. Similarly, finding a saved prefix is beneficial only when retrieving it is preferable to recomputing it under the current resource conditions. Cache hit rate alone does not capture this trade-off.

Sharing retained state across engines. LMCache makes retained KV reusable across requests and compatible inference engines through an independent cache-management layer [16]. Modular connectors let engines store and retrieve state from shared storage backends, while a control interface supports cache orchestration. The same layer also supports state handoff between prefill and decoding workers. This extends reuse beyond one engine's local cache, but requires compatible cache formats, model configurations,

and attention settings. Retrieval and format-conversion costs still determine whether a cache hit saves preparation time.

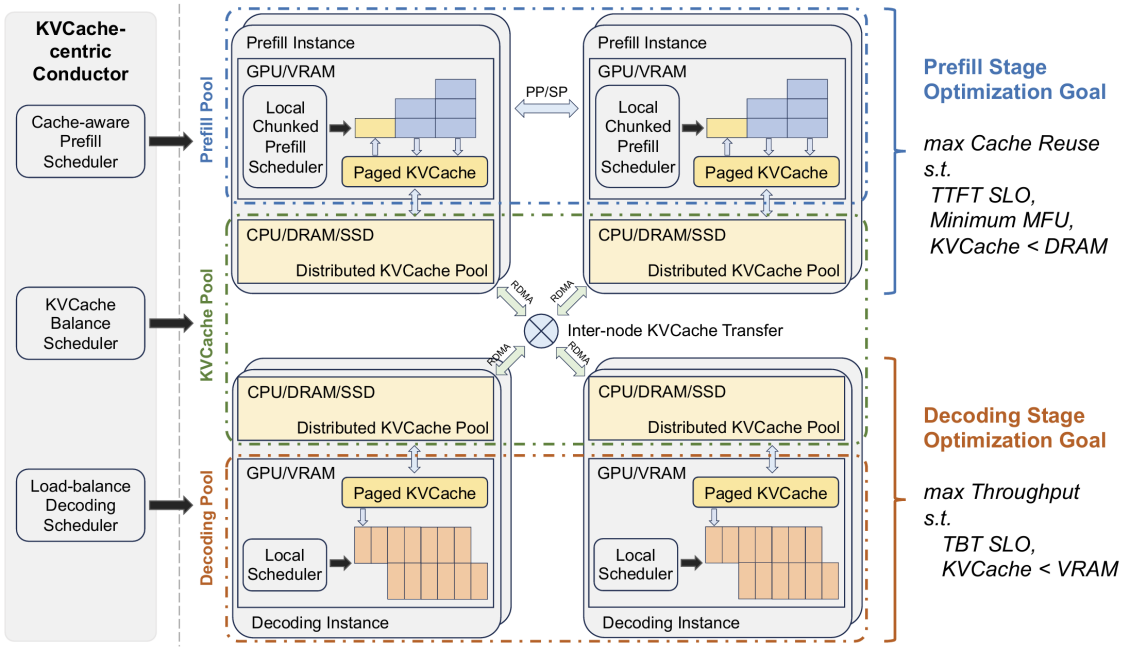


Figure 3.6. The architecture of MoonCake. MoonCake maintains KV-cache pools on DRAM and SSD, retain KV-cache after request finished and reuse it when new request needs it. It also dynamically schedules and transfers KV-cache between GPU nodes.

3.3.3 Cache-aware Execution Coordination

Assigning requests to servers. A server holding a useful prefix may also have a long request queue. Sending every matching request there can increase waiting time despite avoiding computation. Mooncake coordinates request assignment with its global KV-cache [23]. Its scheduler considers reusable prefix length, cache location, and server load when selecting a prefill instance. It can retrieve remote cached state or recompute tokens when transfer is less attractive. The decision therefore balances saved prefill work against queueing and data movement, rather than simply choosing the largest prefix match. Maintaining cache-location information and estimating execution costs add management overhead.

Coordinating cache loading and migration. Strata accounts for cache-loading latency when forming batches and overlaps loading with complementary computation [31]. It

also handles requests that arrive while the same context is still being loaded, avoiding the assumption that every cache hit is immediately ready. Llumnix addresses changes after execution has begun: it migrates requests and their in-memory state between model instances to improve load balance and isolation [26]. Loading-aware scheduling reduces waiting for retained state; live migration relocates active state. Both must weigh the execution benefit against movement and coordination costs.

Coordinating prefill and decoding. KV-cache also connects stages within one request: prefill produces the state that decoding consumes. DistServe assigns these stages to different GPUs to remove their interference, choosing resource allocation and placement with latency requirements and communication bandwidth in mind [40]. Splitwise emphasizes matching each phase to suitable hardware, including heterogeneous machines, to balance throughput, cost, and power [21]. Both introduce a cache handoff before decoding, so separation benefits must outweigh communication and coordination costs. This is execution management within a request, independent of cross-request prefix reuse.

3.3.4 Comparison

Table 3.3 compares the principal decisions. These mechanisms can work together: prefix matching identifies reusable work, retention makes it available over time, and execution coordination determines where and when it is consumed.

Across the three challenges, these techniques primarily reduce repeated state generation or coordinate state availability with execution. Retention increases capacity demand, and restoration consumes bandwidth; approximate passage reuse adds computation and potential quality loss. The useful outcome is the preparation work and waiting time actually avoided, not the amount of cached data or the number of cache hits alone.

Table 3.3. Representative reuse and execution management techniques and their principal trade-offs. Stage separation and migration do not require cross-request reuse.

Method	**Main decision**	**Main cost or limitation**
SGLang [39]	Match and retain shared prefixes	Prefix identity; retained capacity
CacheBlend [33]	Combine cached passages	Selective recomputation; quality risk
Prompt Cache [8]; CacheSlide [15]	Exploit structured prompt segments	Structural assumptions; context correction
Pensieve [35]; CachedAttention [6]	Retain conversation state	Storage occupancy; restoration delay
HCache [7]	Restore KV from hidden states	Projection work; model-dependent size benefit
LMCache [16]	Share retained state across requests and engines	Compatibility; retrieval and conversion costs
Mooncake [23]	Assign requests using cache and load information	Transfer cost; load estimation
Strata [31]; Llumnix [26]	Coordinate loading or relocate active state	Movement and scheduling overhead
DistServe [40]; Splitwise [21]	Place prefill and decoding separately	KV handoff; stage coordination

CHAPTER 4

DISCUSSION AND FUTURE WORK

The techniques reviewed in Chapter 3 address capacity pressure, access bandwidth pressure, and state preparation overhead through different design choices. We discuss their interactions and two directions motivated by agentic applications, in which an LLM makes repeated model calls and uses external tools to pursue a goal: longer contexts within individual requests and longer task lifetimes across requests.

4.1 System-wide Trade-offs and Co-design

Optimizations in one part of the infrastructure change the benefits of others. Quantization reduces storage and transfer volume but adds conversion work; offloading expands capacity but can increase preparation delay. Sparse access saves bandwidth only when the storage layout allows selected entries to be delivered efficiently. Likewise, GQA and MLA reduce the size of KV relative to hidden states, potentially weakening the capacity advantage of restoration methods such as HCache [7]. Benefits therefore depend on the combined design and cannot simply be multiplied across techniques.

Co-design should connect model representations, physical access, and execution management. HiSparse illustrates how model-level sharing of sparse selections can expose future accesses and enable earlier transfers [30]. More broadly, reducing cache size is useful when it improves execution under the relevant resource constraints. Conversion, selection, and restoration costs may shift the dominant bottleneck. The three challenges provide a common basis for examining these interactions, including the quality costs introduced when several approximate methods are combined.

4.2 Ultra-Long Contexts

Complex agentic tasks may require a single request to incorporate extensive documents, code, tool outputs, and reasoning history. Supporting such contexts requires both suf-

ficient capacity and affordable access. With dense attention, every decoding step reads historical KV, so moving more history into a larger storage tier can leave bandwidth as the limiting factor. Future infrastructure should jointly develop compact representations and selective access that control the growth of data movement while preserving long-range dependencies. Native Sparse Attention provides an example of designing sparsity together with training and hardware-efficient execution [36].

An additional challenge is that historical information can become useful much later. An agent may revisit early evidence after obtaining new tool results; low attention at the present step does not establish that this evidence can be discarded. Future designs should distinguish information that need not be accessed now from information that can be removed permanently, and preserve a means of recovering useful history. Task progress and the role of different context segments may help guide these choices. Long contexts also make preparation expensive: incremental construction and reuse should amortize this cost across calls, including when materials are added, reordered, or revised. The objective is efficient and sustained use of long histories, with capacity, access cost, preparation time, and task quality considered together.

4.3 Long-lived Agentic Tasks

Agents may execute for hours or days, alternating model computation with periods of waiting for external tools. Task lifetime therefore extends beyond an individual inference request. During a wait, the context remains valuable although the model is temporarily inactive. Keeping its KV on the GPU consumes capacity that other tasks could use; discarding it makes the next call more expensive to prepare. Future infrastructure should separate the lifetime of retained state from its residence in scarce GPU memory, supporting inexpensive preservation and timely resumption. TokenCake already explores releasing GPU cache capacity during tool calls and preparing state for subsequent execution [2].

Long-lived tasks also require continuity as their histories evolve or execution is interrupted. Context summarization and revision can invalidate previously computed KV, while failures can make extensive reconstruction necessary. Infrastructure should support reuse as the task changes and recovery at a cost appropriate to its accumulated work.

GhostServe illustrates protecting KV for recovery after device failures [9]. KV is nevertheless recomputable model state: resuming an agent also requires its recorded tool results and other task state. Their coordination is essential to preserving the intended context across interruptions. Longer contexts and longer lifetimes reinforce each other, making efficient preservation, access, and restoration central to infrastructure for complete agentic tasks.

CHAPTER 5

CONCLUSION

This survey reviewed KV-cache infrastructure for LLM inference through three responsibilities: representation and compression, storage and access, and reuse and execution management. We analyzed these techniques against capacity pressure, access bandwidth pressure, and state preparation overhead. This framework connects model-level changes with the systems that retain, move, and reuse their cached state, within and across servers.

The reviewed work shows that relieving one bottleneck can increase pressure on another. Compression reduces data volume but may introduce processing costs or quality loss. Offloading expands capacity while making access and restoration more expensive. Reuse avoids repeated computation, but retained state consumes storage and must be compatible with the execution that uses it. The benefit of a technique therefore depends on the model, workload, hardware, and other mechanisms used alongside it. Effective infrastructure requires coordinated choices across these responsibilities.

Future systems must support both ultra-long contexts and long-lived agentic tasks. Longer contexts require affordable access to extensive histories while preserving information needed later. Longer task lifetimes require inexpensive state preservation during tool waits and timely recovery when execution resumes. Together, these trends extend cache management from individual inference requests to complete tasks. Balancing capacity, access, and preparation costs while maintaining task quality will remain central to this development.

REFERENCES

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901. Association for Computational Linguistics, 2023.

[2] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications. *arXiv preprint arXiv:2510.18586*, 2025. Accepted at EuroSys 2027; to appear.

[3] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[4] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. In *Second Conference on Language Modeling*, 2025.

[5] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[6] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference*, pages 111–126. USENIX Association, 2024.

[7] Shiwei Gao, Youmin Chen, and Jiwu Shu. Fast state restoration in LLM serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery, 2025.

[8] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of Machine Learning and Systems*, volume 6, 2024.

[9] Shakya Jayakody, Youpeng Zhao, Chinmay Dhanraj Nehate, and Jun Wang. GhostServe: A lightweight checkpointing system in the shadow for fault-tolerant LLM serving. In *Proceedings of Machine Learning and Systems*, volume 8, 2026.

[10] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. NEO: Saving GPU memory crisis with CPU offloading for online LLM inference. In *Proceedings of Machine Learning and Systems*, volume 7, 2025.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, pages 611–626. Association for Computing Machinery, 2023.

[12] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 155–172. USENIX Association, 2024.

[13] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[14] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. MiniCache: KV cache compression in depth dimension for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[15] Yang Liu, Yunfei Gu, Liqiang Zhang, Chentao Wu, Guangtao Xue, Jie Li, Minyi Guo, Junhao Hu, and Jie Meng. CacheSlide: Unlocking cross position-aware KV cache reuse for accelerating LLM serving. In *24th USENIX Conference on File and Storage Technologies*, pages 83–99. USENIX Association, 2026.

[16] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.

[17] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56. Association for Computing Machinery, 2024.

[18] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344. PMLR, 2024.

[19] Llama Team. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[20] Shutian Luo and Haiying Shen. No buffer, no bottleneck: Efficient zero-copy KV cache offloading for long-context LLMs. In *20th USENIX Symposium on Operating Systems Design and Implementation*, pages 39–54. USENIX Association, 2026.

[21] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.

[22] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. vAttention: Dynamic memory management for serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. Association for Computing Machinery, 2025.

[23] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd*

USENIX Conference on File and Storage Technologies, pages 155–170. USENIX Association, 2025.

[24] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[25] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116. PMLR, 2023.

[26] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 173–191. USENIX Association, 2024.

[27] Hanshi Sun, Li-Wen Chang, Wenlei Bao, Size Zheng, Ningxin Zheng, Xin Liu, Harry Dong, Yuejie Chi, and Beidi Chen. ShadowKV: KV cache in shadows for high-throughput long-context LLM inference. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57355–57373. PMLR, 2025.

[28] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. QUEST: Query-aware sparsity for efficient long-context LLM inference. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 47901–47911. PMLR, 2024.

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[30] Zhiqiang Xie, Zhangheng Huang, Tingwei Huang, Ziyi Xu, Ruiyang Ma, and Christos Kozyrakis. HiSparse: Scaling sparse-attention decoding with hierarchical KV cache management. *arXiv preprint arXiv:2608.07009*, 2026.

[31] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving. In *20th USENIX Symposium on Operating Systems Design and Implementation*, pages 1–16. USENIX Association, 2026.

[32] Amy (Jie) Yang, Jingyi Yang, Aya Ibrahim, Xinfeng Xie, Bangsheng Tang, Grigory Sizov, Jeremy Reizenstein, Jongsoo Park, and Jianyu Huang. Context parallelism for scalable million-token inference. In *Proceedings of the Eighth Conference on Machine Learning and Systems*, 2025.

[33] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25. Association for Computing Machinery, 2025.

[34] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems*, volume 7, 2025.

[35] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery, 2025.

[36] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Yuxing Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23078–23097. Association for Computational Linguistics, 2025.

[37] Tianyi Zhang, Jonah Yi, Zhaozhuo Xu, and Anshumali Shrivastava. KV cache is 1 bit per channel: Efficient large language model inference with coupled quantization. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[38] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[39] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[40] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, pages 193–210. USENIX Association, 2024.